HIL com pyOCD + Semihosting (Framework de Testes)

Estrutura: múltiplos testes + semihosting + automação via pyOCD

1) Registro de testes

```
typedef int (*test_fn_t)(void);

typedef struct {
    const char *name;
    test_fn_t fn;
} test_case_t;

#define TEST(name) int name(void)
#define TEST_ENTRY(fn) {#fn, fn}
```

---

2) Lista de testes (seção dedicada)

```
__attribute__((section(".tests")))
test_case_t test_list[] = {
    TEST_ENTRY(test_add),
    TEST_ENTRY(test_fail),
};
```

Linker script:

```
.tests :
{
    __tests_start__ = .;
    KEEP(*(.tests))
    __tests_end__ = .;
} > FLASH
```

---

3) Runner com semihosting

```
#include <stdio.h>
```

```
extern test_case_t __tests_start__;
extern test_case_t __tests_end__;

static void sh_report(const char *name, int result)
{
    printf("TEST:%s:%d\n", name, result);
}

void test_runner(void)
{
    test_case_t *t = &__tests_start__;

    while (t < &__tests_end__) {
        int r = t->fn();
        sh_report(t->name, r);
        t++;
    }

    printf("DONE\n");
    __BKPT(0);
}
```

---

4) Testes

```
TEST(test_add)
{
    return (2 + 2 == 4) ? 0 : 1;
}

TEST(test_fail)
{
    return 1;
}
```

Execução direta com pyOCD (sem GDB)

Script Python completo

```
from pyocd.core.helpers import ConnectHelper
from pyocd.flash.file_programmer import FileProgrammer
import time
import re

ELF_FILE = "firmware.elf"
```

```python
def run_tests():
    session = ConnectHelper.session_with_chosen_probe()
    session.open()

    target = session.board.target

    # Flash
    programmer = FileProgrammer(session)
    programmer.program(ELF_FILE)

    # Reset e halt
    target.reset_and_halt()

    # Habilita semihosting
    session.options.set("enable_semihosting", True)

    # Resolve símbolo
    elf = session.board.target.elf
    sym = elf.symbol_decoder.get_symbol_for_name("test_runner")
    addr = sym.address

    # Breakpoint final
    target.set_breakpoint(addr)

    # Run
    target.resume()

    output = ""

    while True:
        data = session.get_semihosting_console_output()
        if data:
            output += data

        if "DONE" in output:
            break

        time.sleep(0.01)

    target.halt()
    session.close()

    return parse(output)


def parse(output):
    results = {}
    for name, status in re.findall(r"TEST:(.*?):(\d+)", output):
        results[name] = int(status)
    return results


if __name__ == "__main__":
```

```
if __name__ == "__main__":
    r = run_tests()
    failed = [k for k, v in r.items() if v != 0]
    print("FAIL" if failed else "PASS", failed)
```

Fluxo

1. pyOCD conecta via SWD\
2. Flash do ELF\
3. Reset + run\
4. Firmware executa `test_runner`\
5. Cada teste → `printf("TEST:...")` via semihosting\
6. Python lê buffer semihosting\
7. Detecta `"DONE"`\
8. Parse → PASS/FAIL

Checkpoint objetivo

- adicionar novo `TEST()` → aparece automaticamente na saída\
- saída contém `TEST:<nome>:<status>` para cada caso\
- `"DONE"` sempre presente ao final\
- script Python retorna dicionário `{teste: status}` consistente em execuções repetidas